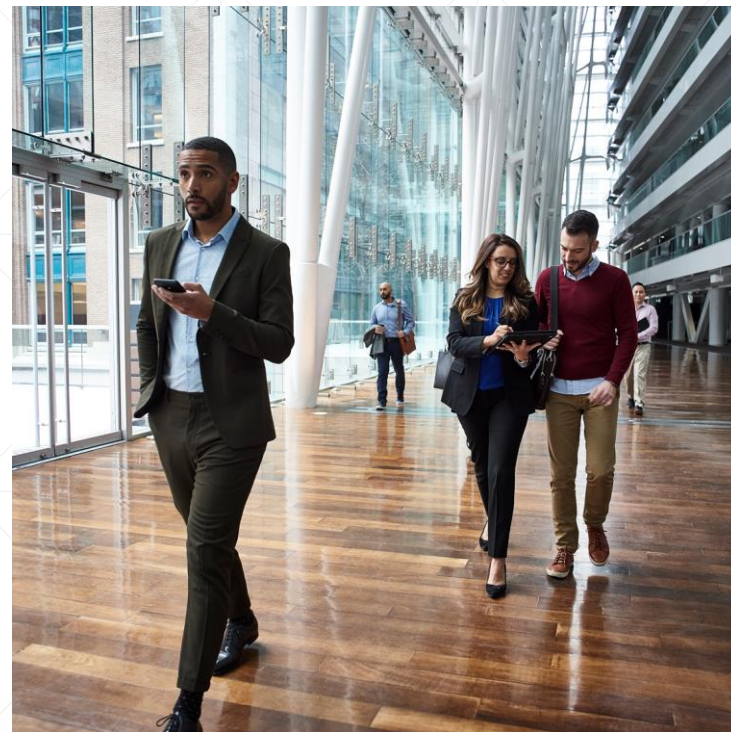


随时可用的线程同步手段



Wait和Notify

北京理工大学计算机学院
金旭亮

引例：线程同步经典编程模型



在多线程开发中，有一个经典的“生产者---消费者”问题：

生产者生产5个数，消费者依次消费5个数，生产者不生产数，消费者需等待，消费者未消费数，生产者需等待。

怎样编写代码实现？

示例运行截图

消费者开始消耗整数.....
生产者开始生产整数.....
生产者设定 (1)
生产者设定 (2)
生产者设定 (3)
生产者设定 (4)
生产者设定 (5)
消费者取走 (5)
消费者取走 (5)
消费者取走 (5)
消费者取走 (5)
消费者取走 (5)

乱套了!

无同步时的情况

ProductNoSyncTest.java

生产者开始生产整数.....
消费者开始消耗整数.....
生产者设定 (1)
消费者取走 (1)
生产者设定 (2)
消费者取走 (2)
生产者设定 (3)
消费者取走 (3)
生产者设定 (4)
消费者取走 (4)
生产者设定 (5)
消费者取走 (5)

工作正常!

有同步时的情况

ProductSyncTest.java

无线程同步条件下的示例代码分析

```
class NoSyncStore {  
    // -1 表示目前没有产品  
    private int product = -1;  
    // 这个方法由生产者线程调用  
    public void setProduct(int product) {  
        this.product = product;  
        System.out.printf("生产者存入 (%d)%n", this.product);  
    }  
    // 这个方法由消费者线程调用  
    public int getProduct() {  
        int p = this.product;  
        System.out.printf("消费者取走 (%d)%n", this.product);  
        return p;  
    }  
}
```

此类是一个“产品仓库”，用于存放产品，在本例中，使用int类型的字段保存生产者生产的一个整数。

这个类没有使用任何的线程同步手段。

生产者线程函数

生产者顺次生产“1、2、...、5”这五个整数，随机休眠特定的时间，然后再把这个整数保存到仓库中。

注意一下仓库对象是从外界注入的，它将被生产者线程和消费者线程所共享。

```
class Producer implements Runnable {  
    private final NoSyncStore store;  
    public Producer(NoSyncStore noSyncStore) {  
        this.store = noSyncStore;  
    }  
    public void run() {  
        System.out.println("生产者开始生产整数.....");  
        // 生产1到5的整数  
        for (int product = 1; product <= 5; product++) {  
            try {  
                // 暂停随机时间  
                Thread.sleep(new Random().nextInt(bound: 10000));  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            // 将整数保存到仓库中  
            store.setProduct(product);  
        }  
    }  
}
```

从外界注入仓库对象

消费者线程函数

消费者代码与生产者代码几乎完全一样，不同之处在于消费者线程是“取”而不是“存”。

```
class Consumer implements Runnable {
    private final NoSyncStore store;
    public Consumer(NoSyncStore noSyncStore) {
        this.store = noSyncStore;
    }
    public void run() {
        System.out.println("消费者开始消耗整数.....");
        // 消耗5个整数
        for (int i = 1; i <= 5; i++) {
            try {
                // 等待随机时间
                Thread.sleep(new Random().nextInt(bound: 10000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            // 从仓库中取走整数
            store.getProduct();
        }
    }
}
```

测试程序

```
public class ProductNoSyncTest {  
    public static void main(String[] args) {  
        NoSyncStore store = new NoSyncStore();  
        // 消费者线程  
        new Thread(new Consumer(store)).start();  
        // 生产者线程  
        new Thread(new Producer(store)).start();  
    }  
}
```

Run: ProductNoSyncTest x

"C:\Program Files\Java

消费者开始消耗整数.....

生产者开始生产整数.....

消费者取走 (-1)

消费者取走 (-1)

消费者取走 (-1)

消费者取走 (-1)

生产者存入 (1)

生产者存入 (2)

消费者取走 (2)

由于没有任何的同步措施，数据的存取是错误的，并且每次运行结果还不一样.....

为了实现生产者与消费者的同步.....



我们需要一种手段来协调生产者线程与消费者线程之间的协作关系。

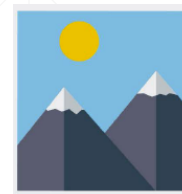


要实现协作，最简单的一种方式就是“等待”与“通知”。

所谓“**等待**”，就是线程在某个地方“暂停”，以等待某些事情的发生。

所谓“**通知**”，就是某些事情的发生之后，向正在等待这个事情发生的线程发布“通告”：“你等所等待的事情已经发生.....”

wait和notify方法



Object类提供了两个方法实现线程之间的“等待”与“通知”：



```
public final void wait() throws InterruptedException {  
    //.....  
}
```



```
public final native void notify();
```



wait方法阻塞本线程，等待其他线程调用notify方法通知自己时才可以继续执行。

线程等待某件事情的发生

```
//在调用wait方法前获得相应对象的内部锁
synchronized(someObject){
    while (等待的事情没有发生){
        //调用object.wait()暂停当前线程
        someObject.wait();
    }
    //代码执行到这里说明等待的事情已经发生
    //执行后继的任务
    doAction();
}
```

通常情况下，使用一个包含共享变量的布尔表达式来表示线程正在等待的“事件”发生与否。

在调用wait方法时，要求线程必须持有对象（someObject）的内部锁。而在调用wait()方法等待时，线程会放弃这个锁，以避免死锁的发生。



如果等待的线程不放弃锁，那么，想发送通知的线程将会由于无法获得锁而无法发出通知，从而引发“死锁”。

“通知”的发送



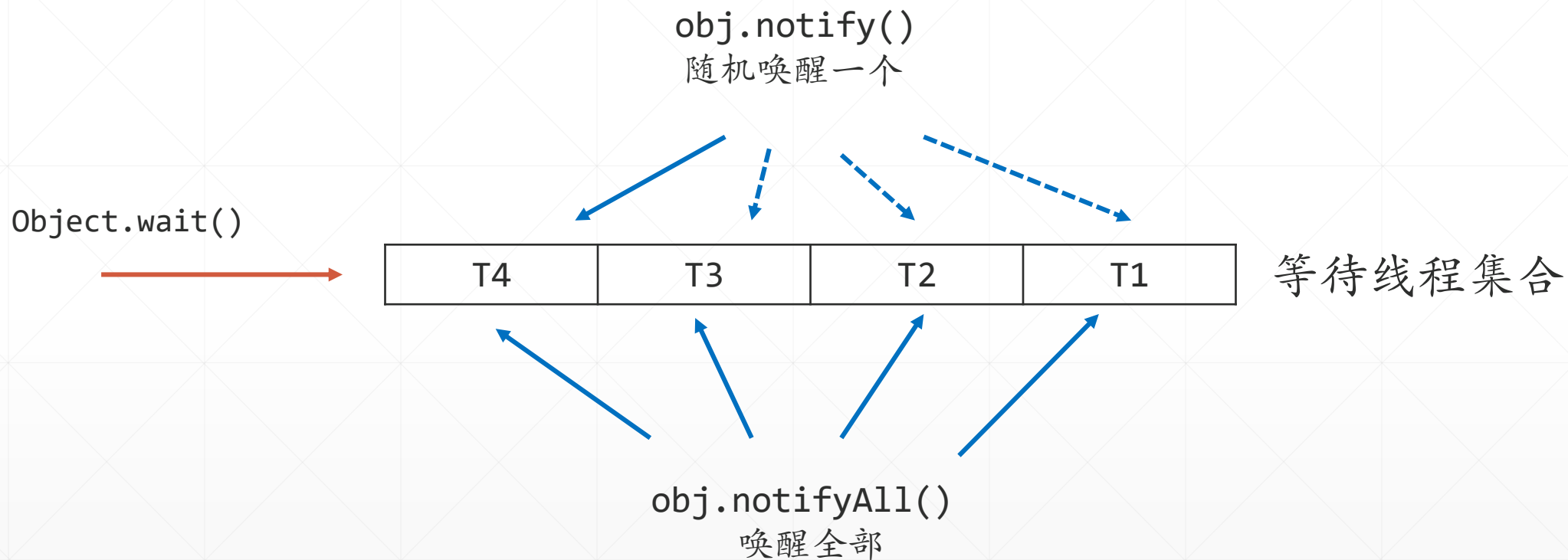
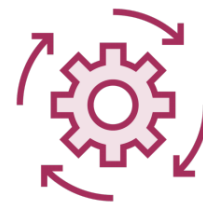
要发送通知，线程必须获得相关对象（someObject）的内部锁，然后才能调用它的notify()方法发送通知。

```
synchronized(someObject){  
    //更新等待线程的所等待事件所涉及的共享变量  
    updateSharedState():  
    //唤醒其他线程  
    someObject.notify();  
}
```



调用wait方法“等待”的线程和调用notify方法“发送通知”的线程，它们调用的wait和notify方法必须针对同一个对象实例（someobject），否则，起不到同步效果。

wait和notify方法的内部机理



Object.wait()/notify()交互协作流程

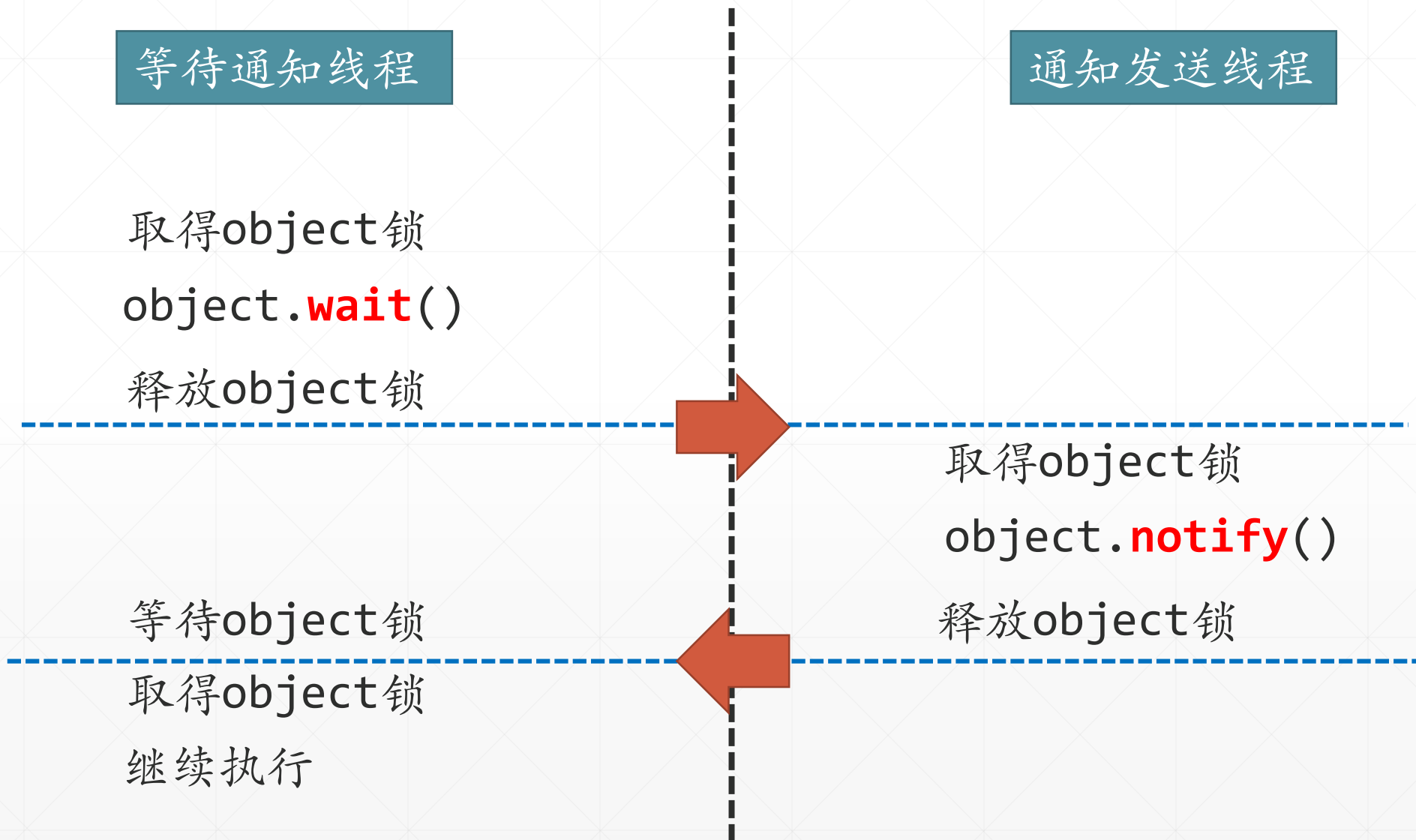
等待通知线程

取得object锁
object.**wait**()
释放object锁

通知发送线程

取得object锁
object.**notify**()
释放object锁

等待object锁
取得object锁
继续执行



通过示例掌握wait/notify用法-1

```
class WaitThread extends Thread {  
    private final Object lock;  
    public WaitThread(Object lock) {  
        super();  
        this.lock = lock;  
    }  
    @Override  
    public void run() {  
        synchronized (lock) {  
            System.out.println("开始 wait time=" + System.currentTimeMillis());  
            try {  
                lock.wait(); //阻塞等待  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            System.out.println("结束 wait time=" + System.currentTimeMillis());  
        }  
    }  
}
```

用于同步线程的关联锁对象，从外部注入以实现多线程共享

此线程启动之后，阻塞等待通知.....

通过示例掌握wait/notify用法-2

此线程启动之后，发送通知.....

```
class NotifyThread extends Thread {  
    private final Object lock;  
    public NotifyThread(Object lock) {  
        super();  
        this.lock = lock;  
    }  
    @Override  
    public void run() {  
        synchronized (lock) {  
            System.out.println("开始 notify time=" + System.currentTimeMillis());  
            lock.notify();  
            System.out.println("结束 notify time=" + System.currentTimeMillis());  
        }  
    }  
}
```

通过示例掌握wait/notify用法-3

```
public class WaitNotifyDemo {  
    public static void main(String[] args) throws InterruptedException {  
        Object lock = new Object();  
        WaitThread t1 = new WaitThread(lock);  
        t1.start();  
        Thread.sleep(3000);  
        NotifyThread t2 = new NotifyThread(lock);  
        t2.start();  
    }  
}
```

两个线程，共享同一个对象是此例的关键。



(1) 如果注释掉Thread.sleep()这句，会发生什么？

(2) 如果NotifyThread先启动，会发生什么？

```
Run: WaitNotifyDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
开始 wait time=1635262032794  
开始 notify time=1635262035803  
结束 notify time=1635262035803  
结束 wait time=1635262035803  
Process finished with exit code 0
```


学以致用.....



基于wait和notify方法，我们可以实现生产者-消费者模式。

“仓库”有重大变化!

```
class SyncStore {  
    // -1 表示目前没有产品  
    private int product = -1;  
  
    // 这个方法由生产者线程调用  
    public synchronized void setProduct(int product) {  
        if (this.product != -1) {  
            try {  
                // 目前仓库没有空间收产品, 请稍候!  
                wait();  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
        this.product = product;  
        System.out.printf("生产者设定 (%d)%n", this.product);  
        // 通知等待区中的一个消费者可以继续工作了  
        notify();  
    }  
}
```

// 这个方法由消费者调用

```
public synchronized int getProduct() {  
    if (this.product == -1) {  
        try {  
            // 缺货了, 请稍候!  
            wait();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
    int p = this.product;  
    System.out.printf(  
        "消费者取走 (%d)%n", this.product);  
    this.product = -1; // 取走产品, -1表示目前仓库手上无产品  
    // 通知等待区中的一个生产者可以继续工作了  
    notify();  
    return p;  
}
```

生产者与消费者代码没有什么变化.....

```
class ProducerSync implements Runnable {
    private final SyncStore store;

    public ProducerSync(SyncStore syncStore) {
        this.store = syncStore;
    }

    public void run() {
        System.out.println("生产者开始生产整数.....");
        // 生产1到5的整数
        for (int product = 1; product <= 5; product++) {
            try {
                // 暂停随机时间
                Thread.sleep(new Random().nextInt(bound: 10000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            // 将产品保存到仓库中
            store.setProduct(product);
        }
    }
}
```

```
class ConsumerSync implements Runnable {
    private final SyncStore store;

    public ConsumerSync(SyncStore syncStore) {
        this.store = syncStore;
    }

    public void run() {
        System.out.println("消费者开始消耗整数.....");
        // 消耗5个整数
        for (int i = 1; i <= 5; i++) {
            try {
                // 等待随机时间
                Thread.sleep(new Random().nextInt(bound: 10000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            // 从仓库中取走整数
            store.getProduct();
        }
    }
}
```

测试代码



```
public class ProductSyncTest {  
    public static void main(String[] args) {  
        SyncStore store = new SyncStore();  
        // 生产者线程  
        new Thread(new ProducerSync(store)).start();  
        // 消费者线程  
        new Thread(new ConsumerSync(store)).start();  
    }  
}
```



Run: ProductSyncTest x

"C:\Program Files\Java\jdk-17\bin\java.exe"

生产者开始生产整数.....

消费者开始消耗整数.....

生产者设定 (1)

消费者取走 (1)

生产者设定 (2)

消费者取走 (2)

工作一切正常!

关于wait(与notify)的使用注意事项



```
private static void throwIllegalMonitorStateException() {  
    try {  
        Object newString = new Object();  
        //当前线程没有锁, 以下这句运行时抛出  
        //IllegalMonitorStateException  
        newString.wait();  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

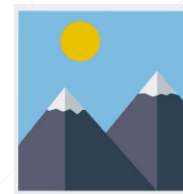


wait()和notify()方法的调用, 都要求拥有锁。

```
//只有wait没有notify, 无限阻塞等待  
private static void blockMain() throws InterruptedException {  
    Object lock = new Object();  
    System.out.println("临界区之前");  
    synchronized (lock) {  
        System.out.println("临界区内部, wait()之前");  
        lock.wait();  
        System.out.println("临界区内部, wait()之后");  
    }  
    System.out.println("临界区之后");  
}
```

MustHasLock.java

小结：明确职责



- ➔ `synchronized`关键字，用于定义临界区，线程互斥访问临界区。
- ➔ `Object`类所定义的`wait()`和`notify()`方法，用于在线程之间发送和接收通知。
- ➔ 两者相互配合，共同协调线程的推进顺序，最终实现线程之间的协作关系，人们将此称为“线程同步”。